User Next Click Prediction Using Machine Learning for UX Analysis

Abstract

Modeling click sequences in a mobile application can support the analysis of user behavior across interface pages and help guide future UX evaluation. This study evaluates next-click prediction in an enterprise application using 46,194 recorded click events in 4,223 sequences. Complete User IDs are assigned to training, validation, and test partitions before prefix–target examples are generated. We compare the proposed validation-selected UNCP 5 with LSTM and Transformer reference models under the same user-disjoint protocol. Mean Top-3 accuracy over three seeds is 85.09% for UNCP 5, 78.00% for LSTM, and 71.92% for Transformer. These results demonstrate next-click prediction under the specified evaluation conditions; potential UX improvements require prospective user testing.

Keywords: User Experience, User Modeling, Machine Learning, Click Models, Neural Networks.

1 Introduction

User interaction behavior on smart mobile devices typically involves a sequence of taps or clicks[1] on touch-screen interface elements, resulting in a series of screen state transitions in response to these actions [1]. Predicting the next user click through machine learning models opens up new possibilities for enhancing user experience. For instance, interface elements can be reordered based on user interaction frequency, thereby improving efficiency and speeding up task completion. Furthermore, Adaptive User Interfaces (AUIs) can be dynamically activated to personalize the layout and make the interface more intuitive for future use [2]. An AUI adapts the arrangement of UI elements to the user's preferences or behavior patterns. This means components such as surveys, fund transfers, settings, and events on a page can be reordered either by the user or automatically based on usage frequency [3, 4].

Numerous studies have focused on modeling user click behavior across domains such as mobile interaction [5], web navigation [6], advertising [7], news consumption [8], and e-commerce. Several works [1, 4, 9] have specifically explored user performance

[1] In this study, the term *click* is used instead of *tap* to refer to interactions on the mobile application screen.

prediction at the application level. Zhou and Li [10] investigated computational modeling of click behavior as a foundation for adaptive UI systems. Their work focused on predicting the next UI element a user is likely to click. They developed LSTM- and Transformer-based learning models trained on user click data, taking into account click history, screen content, and current time to accurately forecast the next user action.

Our Contribution: The main contributions of this study are summarized as follows:

- Model comparison: Evaluation of transparent baselines, LSTM, GRU, Transformer, and six UNCP variants on the same held-out-user partition; the variants and their component combinations are identified in the result tables.
- Real dataset: Analysis of 46,194 click events in 4,223 sequences from 52 recorded User IDs, with complete User IDs held out from model fitting and preprocessing.
- UX techniques: Use of predicted click rankings to identify possible UX improvements for evaluation in controlled usability studies.
- Decision support: Examination of how predicted click rankings may help prioritize interface flows and possible design improvements for later UX investigation; efficiency and usability effects are not measured in this study.

2 Related Work

In recent years, extensive research has been conducted on modeling user interactions and enhancing user experience through predictive systems. In this section, we review key studies that have contributed to this area.

Baeza-Yates et al. [1] proposed a predictive framework that identifies the next mobile application a user is likely to engage with. Their model utilizes a set of contextual features to predict the upcoming app interaction and subsequently reorganizes the app's menu layout based on the frequency of user access, thereby offering a more personalized experience. Chen et al. [11] made significant advancements in predicting click-through rates (CTR) in Taobao's recommendation system by employing deep learning techniques. They introduced a Transformer-based recommendation model that represents user and item features as embedded vectors. These vectors are processed through a multi-layer fully connected neural network to estimate the likelihood of a user clicking on a specific item. Qin et al. [12] addressed user interest modeling by employing sequential behavior analysis in mobile applications. Their approach first selects relevant user actions from historical interaction sequences using information retrieval techniques, and then feeds them into a deep learning model for CTR prediction. This combination of retrieval-based filtering and deep learning improved the model's capacity to capture nuanced user preferences over time.

Yang et al. [13] simulated user experience in the process of running a mobile app in their proposed model using machine learning. They used predicted app pages with the effect of time on element selection (see figure 2) to train neural networks, which were formed from click information of all users during a specific task process. Zhou and Li [10], researchers at Google, designed a deep learning model based on Transformer using a dataset obtained from user interactions with a mobile application. This model predicts the next element that the user is most likely to click on, based on the user's

click history and user interface information (see figure 1). Wu et al. [14] concentrate on understanding the functionality of user interface elements. This method enables better analysis by mapping UI elements to previous examples, incorporating their semantic properties, appearance, text, and how they function. The model is used for generating in-app tutorials, searching for UI elements, and testing them. For example, this algorithm can identify that a UI element on a page is a button with the text 'Login' or a text field with the text 'Sign Up'.

Sun et al. [15] introduced a new GUI-based architecture (GUITOD) that is capable of directly interacting with software using graphical tools instead of complex textual commands. The operations performed include clicking on buttons, text fields, search fields, and similar elements. Ultimately, by clicking on these elements, actions are carried out based on the scenario and user flow. Additionally, it is used for training dialogue models in mobile applications. This model also predicts the next action of the user. It is widely used for performing specific tasks such as hotel reservations, calendar scheduling, and online shopping. Wu et al.[16] trained machine learning models to predict semantic information about user interfaces to make mobile apps more accessible and automated. They introduced a Never-ending UI Learner model, which automatically extracts real apps by interacting with UI elements. This model helps improve the understanding of UI elements in the app. Feng et al.[17] have made mobile app tutorial videos a popular method for users to learn various features. To make these videos easily understandable for users, video creators need to annotate different actions in the video. They introduced an approach called Video2Action, based on image processing and deep learning, which automatically predicts and highlights the click locations in the video. Feng et al.[18] introduced a novel approach to automatically extract UI data from Android apps using Large Language Models (LLMs). This model is used for UI modeling and element identification. Zhou et al.[19] proposed a two-stage long-term sequence modeling approach, consisting of offline extraction and online inference, which is used to familiarize with user interests for predicting associations with target items.

Related work on intelligent optimization provides additional systems context. Addula et al. [20] proposed a hybrid Kookaburra–Pelican optimization approach for dynamic cloud load balancing, targeting improved workload distribution and resource utilization. Although that study addresses infrastructure optimization rather than next-click prediction, it illustrates how hybrid optimization can support adaptive system decisions. We cite it as broader background only, not as evidence for predictive accuracy or UX improvement in the present study.

Human-centered AI is also relevant when predictive systems are deployed in workplace settings. Meduri et al. [21] studied how personalization, employee training, and feedback shape the effects of AI-supported workload management and emphasized that human-centered implementation matters beyond model performance. We cite this work as contextual motivation for careful, human-centered UX analysis; it is not used as evidence that the present next-click model improves employee well-being or usability.

As reviewed in our Related Work section, prior studies have examined next-action prediction and adaptive interfaces. The present study contributes a leakage-aware

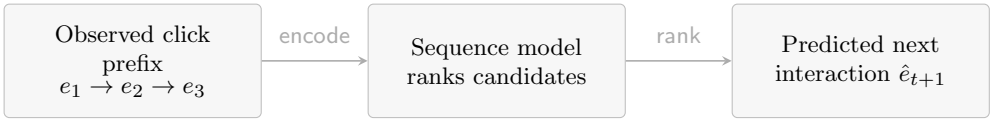


Fig. 1: Redrawn schematic of next-click prediction from an observed click prefix, inspired by the task formulation in Zhou and Li [10].

comparison of next-click models on an enterprise clickstream and explores predicted rankings as a possible decision-support signal for UX researchers. We do not claim that the model automates UX evaluation or improves usability; those outcomes require comparison with established UX methods and direct user-study evidence.

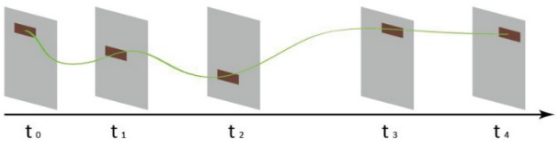


Fig. 2: User pages and clicks at t_i or different times, showing how the user clicked on elements at different times. For example, the user clicked on the first element at time t_1, the second element at time t_2, and the third element at time t_3 [13]

.

3 Proposed Model

The main objective of this study is to predict the next user click and examine how the resulting rankings could support UX analysis. This section describes data collection and preprocessing, followed by the evaluated sequence models (see Figure 3). The resulting rankings can guide proposed design improvements; the present study does not establish their effects through first-click or usability tests.

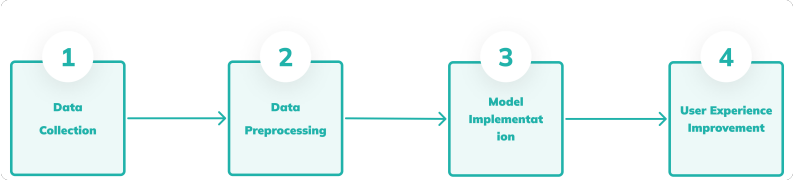


Fig. 3: Steps for implementing and evaluating the next-click prediction models.

Each page contains several clickable elements (see Figure 4). When a user clicks on one of these elements, an event is recorded, and the user is redirected to another page with a different set of elements. For example, the *Notifications* page includes a *Back* button that allows the user to return to the previous page. Similarly, the *Home* page

features clickable elements such as *Users*, *Market*, *Settings*, and *Notifications*, each of which navigates the user to a different page upon selection.

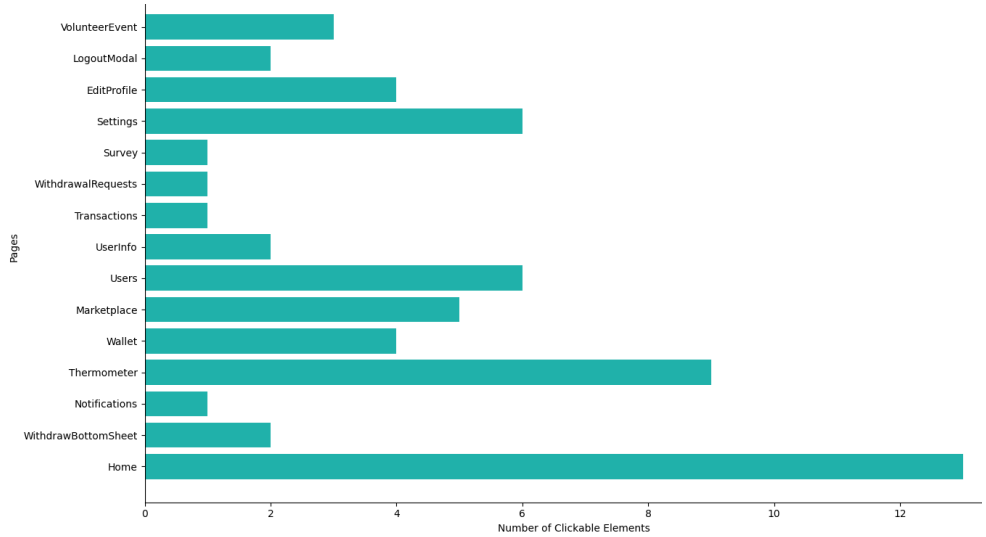


Fig. 4: The number of clickable elements on each page is shown in this chart. For example, Settings page contains 6 clickable elements, while the Notifications page has 1 clickable element.

Data was collected via a mobile employee recognition app, using the Metrix[2] platform to label clickable elements. User click sequences were then constructed in preprocessing by concatenating recorded click events from standard users. The platform recorded an opaque, UUID-formatted MX User Id for each User ID; a consistent numeric replacement code was used for sequence reconstruction and user-disjoint evaluation. Neither identifier was a predictive model input.

3.1 Data Preprocessing

Click events were ordered by recorded User ID and event time where timestamps were valid. The primary analysis retains consecutive clicks on the same element because repetition may reflect delayed feedback, confusion, accessibility difficulty, or frustration. The analytic dataset contains 46,194 click events in 4, 223 sequences from 52 recorded User IDs. Session construction and its sensitivity to inactivity thresholds are described below.

- This method defines a user's click sequence from the first click until a set expiry time typically two minutes of inactivity. If no click occurs within two minutes, the sequence from the first to the last click is stored (Figure 5). The 120-second and 240-second inactivity cutoffs are evaluated as alternative session-construction settings.

[2] metrix.ir

They are not assumed to be universal mobile-session durations, and no claim of statistical equivalence is made.

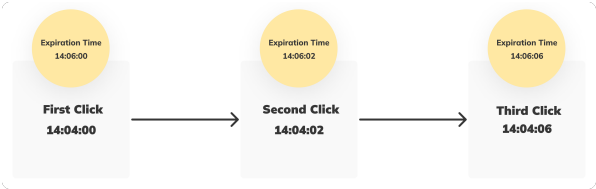


Fig. 5: Illustration of the 120-second inactivity cutoff evaluated for click-sequence construction.

- Sequence Based on User App Sessions: A sequence was defined from the moment the user opened the app until it was closed. In this scenario, the user opened the app, clicked on various parts, and then closed the app, which was recorded as a click sequence.

The two cutoffs provide a sensitivity check around short mobile interactions rather than a literature-established optimum. In the repeat-inclusive export, 4,185 rows (44,823 clicks) have aligned timestamps for within-row gap analysis. Splitting these rows at gaps greater than 120 and 240 seconds yields 4,306 and 4,263 sequences, respectively, without changing the 44,823 observed clicks. Mean sequence lengths are 10.41 and 10.51 clicks, respectively (median 5; 95th percentile 25 in both settings). The corresponding numbers of next-click targets are 40,517 and 40,560. This analysis excludes rows with unusable timestamp alignment; it does not reconstruct the complete original event log or establish that either cutoff is optimal. The event-table fields are illustrated below (Figure 6).

- Column 1: User ID
- Column 2: Page
- Column 3: Clickable Element
- Column 4: Date and Time

ID	Click sequence	Time (Gregorian calendar)
4	marketplace, users, settings	2023-10-09 00:08:03; 00:08:06; 00:08:09
5	volunteerEvent, marketplace, dashboard, settings	2023-10-16 14:38:07; 14:39:53; 14:39:54; 14:39:57
7	close, volunteerEvent, settings, withdrawRequests, transactions	2023-10-17 08:34:25; 08:34:47; 08:35:00; 08:35:38; 08:35:40
8	dashboard, settings, notifications, backNotifications	2023-10-17 13:56:01; 13:56:53; 13:56:58; 13:57:01
4	close, users, dashboard, logout, okLogout	2023-10-17 14:41:03; 14:41:04; 14:41:07; 14:41:09
40	excellent, normal, submitThermometer, surveysBox, notifications, backNotifications	2023-10-30 07:36:22; 07:36:22; 07:36:22; 07:36:26; 07:36:33; 07:37:07
5	close, marketplace, users, dashboard, logout, notifications	2023-10-31 21:20:54; 21:20:57; 21:20:58; 21:21:08; 21:21:09; 21:21:10

Fig. 6: Examples of user click sequences recorded during app interaction. Dates are shown in the Gregorian calendar; times are retained from the event log.

For an event to be considered valid, it must be clickable, visible, and enabled. Some elements may be clickable but inactive, which would still qualify them as verified clickable elements. Repeated clicks are retained in the revised primary analysis. The final analytic dataset contains 46,194 clicks, 4,223 sequences, 52 recorded User IDs, and 93 literal click labels. The User IDs are used for group splitting and are not model inputs. Transitioning from one page to another can occur in several ways:

- A new page is opened.
- A bottom sheet menu is displayed.
- A modal or dialog box is shown.

3.2 Data Modeling

To model the data, a dataset of user click sequences is obtained after preprocessing. These click sequences are initially performed by a user with a defined expiration time, and the user identifier is recorded in the first column. The second column contains the name of the page where the element was clicked. Subsequently, the user's click sequence across different pages is displayed, with the clicked element in each page documented accordingly. Finally, the date and time of these click sequences are recorded. The event table supports sequence construction and retrospective diagnostics; the evaluated predictor receives only the observed click-label prefix, not the User ID, page labels, or timestamps.

3.2.1 Sequence Lengths & Time Spans

After restoration, sequence length ranges from 1 to 1,156 clicks (median 5; mean 10.94). Model context is capped at the last 32 observed clicks, so extremely long repeated-click runs do not increase the input length beyond the prespecified context window.

4 Architectures for Predicting the Next User Click

Following the data collection and preprocessing steps previously outlined, the architectures of two models developed for predicting the next user click are presented in the sections below.

4.1 Modeling User Click Behavior with LSTM

A click event is characterized by attributes such as the page, the element clicked, and the time the click occurred. The i-th event in the click sequence is denoted by e_i and is defined as shown in (Equation 1):

$$e_i = [s_i, c_i, t_i] \tag{1}$$

where s_i represents the page in the i-th event, c_i denotes the element that was clicked, and t_i indicates the time of the event. The page s_i consists of several clickable elements $c_i \in s_i$, where $1 \leq i \leq |s_i|$ and $|s_i|$ represents the number of clickable elements

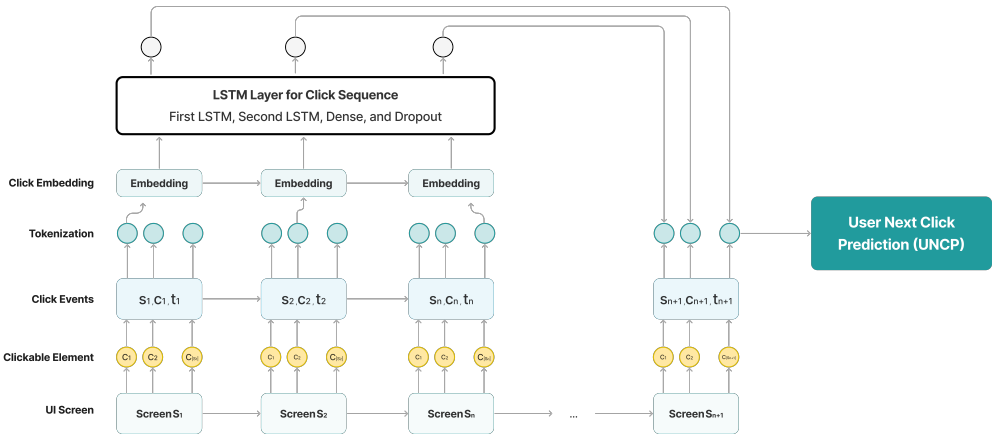


Fig. 7: The architecture of the LSTM model, comprising layers for pages, clickable elements, user click events, tokenization, embedding, and LSTM layers.

on the page. Therefore, the click sequence of a user is defined as the set $e_{i:n}$.

$$e_{i:n} = \{(s_1, c_1, t_1), (s_2, c_2, t_2), \ldots, (s_n, c_n, t_n)\}$$

The recorded events form a click sequence (Figure 7), but the revised predictive models tokenize only the observed clicked-element labels. Page and time remain in the event archive for reconstruction and retrospective analysis; neither is a model feature. The token map is fitted on the training partition and maps an unseen input label to an unknown token (Equation 2).

$$v_i = \text{tokenizer}(c_i) \tag{2}$$

Then, during embedding, the generated numerical codes are mapped to fixed-dimensional vectors so that these vectors can capture the semantic relationships between words and be used for learning (Equation 3).

$$embedding(v_i) = v_i \cdot w_e \tag{3}$$

In this, w_e is the matrix that holds the embedding weights, where $w_e \in \mathbb{R}^{V \times D}$, and its dimensions depend on the number of words in the vocabulary and the embedding dimensions. V is the number of unique words in the dataset, and D is the dimensionality of the embedding vector, which is a hyperparameter typically set as the number of embedding dimensions. The first LSTM layer processes the input sequence, models the temporal relationships between the clicks, processes a sequence of vectors v_i, and outputs the hidden state h_t (Equation 4).

$$h_t = l_{lstm} \rightarrow embedding(v_i) \tag{4}$$

Where $h_t \in \mathbb{R}^M$ represents an M-dimensional space. In the model, this means that the output of each node or unit in this layer will be an M-dimensional vector of real numbers. In other words, if M equals 128, each output vector from this layer will have 128 components. For example, in the LSTM layer in the model architecture described, if M is set to 128, each hidden state (denoted as Hidden State) output from this layer will be a vector of real numbers representing the information the model acquires from the data at each step. The revised LSTM reference implementation has one 64-unit recurrent layer; the final non-padding hidden state represents the observed prefix. The *Dense* layer, a fully connected layer, converts the output of the previous layers into a fixed-dimensional vector (Equation 5).

$$l_{dense} : dense(h_t) \tag{5}$$

The *Dropout* layer randomly disables some neurons during training to prevent overfitting and make the model more generalizable (Equation 6).

$$l_{dropout} : dropout(l_{dense}) \tag{6}$$

The prediction task is defined in Equation 7. At inference time, the model has access only to the observed click labels up to step n; it ranks next-click labels from the training vocabulary.

$$\hat{\mathbf{p}}_{n+1} = f_\theta(c_{\max(1,n-L+1):n}), \qquad \hat{c}_{n+1} = \arg\max_c \hat{p}_{n+1}(c), \quad L = 32 \tag{7}$$

Here $c_{\max(1,n-L+1):n}$ is the last $L = 32$ observed click labels (or the shorter available prefix), f_θ returns a probability distribution over next-click labels, and $\hat{c}_{n+1}$ is its highest-ranked label. The actual next click c_{n+1} is the evaluation target. The destination page s_{n+1} and event time t_{n+1} are known only after that event and are never supplied to f_θ. Recorded destination pages are used solely for retrospective page-level diagnostics.

4.2 Modeling User Click Behavior with Transformer

First, a semantic representation of the clicked element in each click event is extracted to better understand the features of the elements and the user's click behavior. Then, a sequence of click events is modeled to encode the user's click history. Finally, based on the encoded click history, the reference model ranks next-click labels in the training target vocabulary (Figure 8). It is not given the future destination page or a page-specific candidate mask.

The Transformer uses the same observed-click tokenization as the LSTM reference model (Equation 2). In the following, the focus shifts to the embedding layer.

Subsequently, in the embedding step, the generated numerical codes are transformed into fixed-dimensional vectors so that these vectors can capture the semantic relationships between the elements and be used for learning (Equation 8).

$$\text{Embed}_i = \text{Embed}(c_i) \tag{8}$$

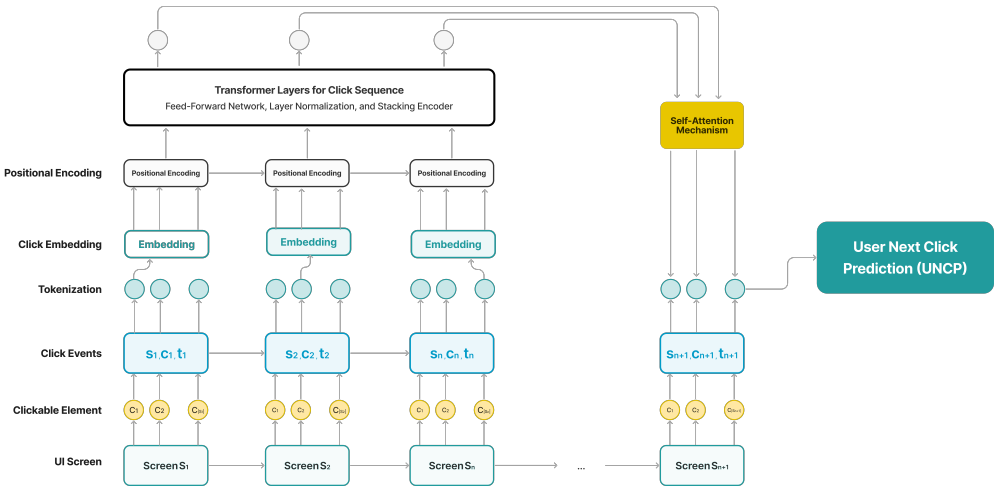


Fig. 8: The Transformer architecture, which includes layers for pages, clickable elements, user click events, tokenization, embedding, and Transformer layers such as feed-forward neural networks, normalization, and stack-based encoders.

Only the observed click label is embedded. The page and timestamp are retained in the event data for retrospective analyses, not concatenated with the model input.

[]

In the next step, positional encoding is added to the embedded vector to preserve the order of clicks in the sequence, as the Transformer model inherently does not encode the sequence order (Equation 9).

$$Embed'_i = Embed_i + PositionalEncoding(i) \tag{9}$$

The position-aware embeddings are projected with learned matrices W_Q, W_K, and W_V to form the query Q, key K, and value V representations. Scaled dot-product attention applies a row-wise softmax to the query–key scores and uses the resulting weights to combine the values (Equation 10).

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{10}$$

To compute the dot product between the matrices Q and K, the calculation QK^T is performed first. Here, K^T is the transpose of the matrix K, where its rows and columns are swapped. This allows the dot product between each Query and each Key to be computed. For normalization, the values obtained from the dot product QK^T are divided by $\sqrt{d_k}$, where $\sqrt{d_k}$ is the dimensionality of the key. This helps reduce fluctuations in large values and makes the model train more effectively. The reason for using $\sqrt{d_k}$ is that as the number of dimensions increases, the dot product values become larger, and dividing by $\sqrt{d_k}$ helps balance these values. Next, a fully

connected neural network x is applied independently to each position in the sequence and normalized to improve the stability of the training process. In the next step, the remaining connections are added to the network to prevent the vanishing gradient problem. The encoder layers, which include self-attention and fully connected neural networks, are repeated multiple times. This repetition allows the model to learn more complex datasets (Equation 11).

$$H = StackedEncoderLayers(E') \tag{11}$$

The repetition of encoder layers is considered one of the key principles in the Transformer model. This means that instead of using a single encoder layer, multiple encoder layers are stacked. This approach allows the model to learn more complex and higher-level features from the data. It is indicated as the final output after passing through all the encoder layers. These vectors represent higher-level representations of the inputs, which can be utilized for subsequent stages such as computation and predicting the next click. In the next step, the final hidden state, which is the output of the stacked encoder layers, is passed through a $softmax$ layer to produce a probability distribution, $Prob_{n+1}$, over the potential next clicks (Equation 12).

$$Prob_{n+1} = Softmax(H) \tag{12}$$

Finally, the Transformer model is used to predict the probability of which element on the screen, c_{n+1}, the user is likely to click on in the $(i+1)$-th step, and to determine the function f. The $argmax$ function returns the position or index with the highest probability value (Equation 13).

$$UNCPc_{n+1} = argmax(Prob_{n+1}) \tag{13}$$

4.3 Proposed UNCP Hybrid

The proposed User Next Click Predictor (UNCP) preserves the click-sequence input used by the preceding models. A shared 32-dimensional click embedding feeds two parallel branches: a 64-unit GRU for short transition patterns and a two-layer, four-head Transformer for longer contextual dependencies. The final real-token Transformer state is projected to 64 dimensions. A learned sigmoid feature gate mixes the GRU and Transformer states elementwise; layer normalization and dropout precede the global click classifier. Neither future clicks, user identity, page identity, nor timestamps are model inputs. The individual operations are established building blocks; the contribution claimed here is their task-specific gated combination and its leakage-aware evaluation, not universal or first-ever architectural novelty.

UNCP 2 graph-aware extension.

UNCP 2 constructs a weighted, undirected click-transition adjacency from encoded training prefixes only, adds self-loops, and applies symmetric degree normalization. One graph propagation step transforms the shared 32-dimensional element embeddings; the resulting graph-aware token vectors feed a 64-unit GRU and a two-layer, four-head Transformer. A masked mean of observed graph token vectors forms a third

context state. A learned feature gate combines the GRU state with the average of the Transformer and graph states, followed by layer normalization, dropout, and global classification. The validation and test clicks do not enter graph construction. This is a task-specific combination of established graph, recurrent, and attention operations; the observed advantage is limited to the reported dataset and split.

UNCP 3 and UNCP 4 exploratory variants.

UNCP 3 combines a GRU, Transformer, and a plain-PyTorch structured state-space duality (SSD) recurrence inspired by Mamba-2; it does not use the official optimized Mamba-2 kernel. UNCP 4 combines typed page–element graph attention, a training-only transition graph, a GRU, an adaptive fusion gate, and candidate masking. Its mask uses the last observed click's inventory page as a proxy for the current page, augmented with training-observed successors; 33 of 93 click-label strings lack an exact inventory match and use an unrestricted fallback. Neither variant improved on UNCP 2 in the held-out-user experiment, so they are retained as transparent exploratory comparisons rather than selected as the final predictor.

UNCP 5 adaptive transition fusion.

UNCP 5 combines the trained UNCP 2 probability vector with a smoothed back-off transition distribution over the last one to four observed clicks. Transition counts and the global smoothing prior use training examples only. The transition weight is a sigmoid function of log transition support and the difference between the two predictive entropies; the final probability vector is their convex combination. We selected the three gate coefficients from a fixed grid of 76 rules by mean validation Top-3 across seeds 11, 23, and 37, breaking ties by validation Top-1 and negative log-likelihood. Only after freezing that choice did we evaluate the held-out users. Thus UNCP 5 is an adaptive probability-level ensemble, not a newly trained end-to-end neural architecture. No future page, timestamp, click, or test label is used to build the transition model or select its gate.

UNCP 6 repeat-aware dual-scale model.

One GRU encodes the complete observed click prefix. A second GRU encodes consecutive runs after repeated labels are compressed, with each run length embedded as a feature. A learned gate combines the two states. The next-click distribution factorizes into a repeat-versus-switch probability and a classifier over elements conditional on switching; the repeat option is the last observed label. No future click is used in run construction. Three fixed-seed fits on the same user-disjoint split reach 84.08% mean Top-3, below UNCP 5's 85.09%. On the 1,541 repeated-click targets, their Top-3 values are 94.09% and 94.07%, respectively; on 9,292 switch targets they are 82.42% and 83.60%. This diagnostic prevents a gain on repeats from being mistaken for better prediction of changed actions.

4.4 Potential Decision Support for UX Analysis

The evaluated models rank likely next clicks from previously observed click sequences. These rankings may help UX researchers identify recurrent transitions, unexpected

paths, and interface elements that merit inspection. They do not measure user intent, task success, cognitive load, satisfaction, or usability, and they are not equivalent to conventional UX testing. Whether they reduce evaluation time or effort must be established in a prospective comparison with an existing UX workflow.

- **Supporting inspection of interaction flows:** Observed click sequences and predicted next-click rankings can be used to flag common or unexpected paths for later review. The model cannot determine from clickstream data alone whether a task was completed correctly or why a user selected an element. Figure 9 illustrates the prediction task rather than a completed usability test.

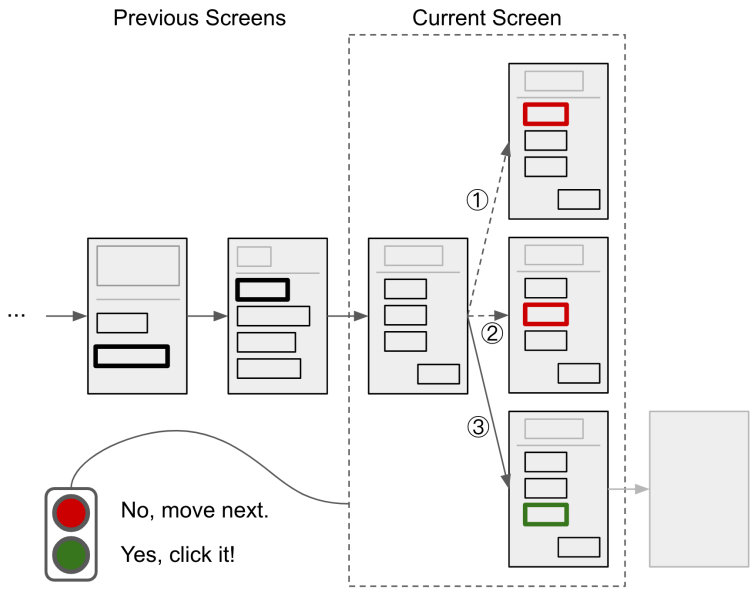


Fig. 9: Next-click prediction uses an observed click prefix to rank possible subsequent interactions [10].

- **Supporting first-click analysis:** Initial-click frequencies may indicate which elements attract interaction and can help select screens for a formal first-click study. They do not establish whether a first click is correct for a task or explain the user's intent [22].
- **Supporting information-architecture review:** Ranked click patterns may suggest changes to element prominence and navigation. They are not a substitute for card sorting, which directly examines users' conceptual groupings and labels [23].

4.5 Evaluation Metrics

Evaluation metrics are used to assess the performance of different models. These metrics help determine how well the models have performed and whether they can make accurate predictions. Several metrics are calculated to evaluate the quality of predictions for each method, which are listed below.

The primary test holds out complete User IDs. We assigned all sequences of each recorded User ID to exactly one of the training, validation, or test partitions before creating overlapping click-prefix examples. The 46,194 figure counts click events, whereas the split counts below are supervised next-click targets. A sequence of length L produces $L - 1$ targets; therefore, 46,194 events across 4,223 sequences produce $46{,}194 - 4{,}223 = 41{,}971$ targets. The fixed split contains 32, 9, and 11 User IDs, respectively ($32 + 9 + 11 = 52$), and yields 25,399 training, 5,739 validation, and 10,833 test targets ($25{,}399 + 5{,}739 + 10{,}833 = 41{,}971$). Automated protocol checks confirmed that no User ID or source sequence occurs in more than one partition. The vocabulary, popularity counts, transition frequencies, and smoothing priors are derived only from training data. Validation data select checkpoints and the UNCP 5 fusion rule; the test partition is evaluated after those choices are fixed. These checks and the split manifest accompany the executable experiment files. For the "Uniform expected" baseline, each test target is assigned equal probability over the evaluation candidate vocabulary. If the candidate set has size K, sampling uniformly without replacement gives an expected Top-1 accuracy of $1/K$ and an expected Top-3 accuracy of $\min(3, K)/K$. The reported 1.10% Top-1 and 3.30% Top-3 values therefore correspond to the 93-label global candidate vocabulary (1/93 and 3/93, rounded to two decimals). This analytical expectation is not estimated from random draws. Page-conditioned uniform diagnostics are reported separately because their candidate sets and page availability are distinct from the global-label task.

4.5.1 Top-K Accuracy:

In this metric, the model's accuracy is calculated based on the number of times the target element appears among the top K predictions of the model. For example, to compute 'Top-1 Accuracy' and 'Top-3 Accuracy,' the number of times the target element appears within the first and third predictions of the model, respectively, is reported. The higher the value of this metric, the better, as it indicates that the model has performed better in identifying the target elements.

4.5.2 Absolute Ranking Position:

This metric refers to the ranking position of the target element among all predictions. The lower the value of this metric, the better, as it indicates that the target element is placed in a higher position among the predictions.

4.5.3 Relative Ranking Position:

A relative ranking metric divides the target's absolute rank by the number of candidate elements. Lower values indicate a better rank. This metric complements Top-K accuracy by describing the target's position relative to the candidate-set size, as discussed in [10].

In addition to micro Top-1 and Top-3 accuracy, we report macro averages across held-out User IDs and target labels, results by recorded page and by training-label frequency group, and candidate-set diagnostics. Neural models are trained with seeds 11, 23, and 37; tables show the mean and sample standard deviation across these runs. For selected paired comparisons, we resample held-out Users as clusters to obtain exploratory confidence intervals and use paired User-level tests with multiplicity correction. Page labels recorded with the eventual click are used only for retrospective diagnostics, not as inference-time inputs. Within-row 120- and 240-second gap refinements and a collapsed-repeat variant assess preprocessing sensitivity. On the same grouped-User-ID assignment, the first-order Markov Top-3 scores for the two gap settings are 65.07% and 65.10%, and the fourth-order back-off Markov scores are 79.39% and 79.48%, respectively (10,798 and 10,804 test targets). These are descriptive comparisons on changed target sets, not statistical evidence of equivalence or causal threshold effects. The LSTM and Transformer scores reported for the earlier, smaller aligned-event reconstruction are not attributed to this repeat-inclusive sensitivity analysis.

4.6 Data Preparation

The original implementation expanded 4,216 recorded sequences into overlapping prefix–target examples and then randomly split those examples. A reconstruction showed that 6,061 of 6,215 historical test examples shared their source sequence with training, and all 52 recorded User IDs occurred in both partitions. The historical 88.91% LSTM Top-3 result is therefore retained only as context and is not used as evidence of unseen-user generalization. After repeated-click restoration, the revised dataset contains 46,194 click events in 4,223 sequences. Because the first event of each sequence supplies context but has no preceding click from which to form a target, these data produce $46{,}194 - 4{,}223 = 41{,}971$ next-click targets.

For the revised primary evaluation, complete User IDs are assigned before prefix generation: 32 IDs for training, 9 for validation, and 11 for testing, yielding 25,399/5,739/10,833 next-click targets. The User ID counts sum to 52 and the target counts sum to 41,971, exactly matching the totals above. Vocabulary and frequency statistics are fitted on training data only, validation loss selects checkpoints, and targets unseen during training are scored incorrect. Neural models are run with seeds 11, 23, and 37. A secondary chronological analysis uses only rows with aligned, valid timestamps; because its cohort differs, it is reported separately rather than interpreted as an improvement over the user-disjoint test.

Implementation and model selection.

The fitted sequence models use 32-dimensional click embeddings. The LSTM and GRU baselines each have one recurrent layer with 64 units. The Transformer baseline has two encoder layers, four attention heads, 64-dimensional ReLU feed-forward layers, sinusoidal positional encodings, residual connections, and post-residual layer normalization. Dropout is 0.2 within the Transformer and before the final classifier. Each input contains at most the last 32 observed clicks; shorter prefixes are padded on the right. Transformer attention masks padding; the baseline recurrent and Transformer classifiers read the final real prefix-token state, while the hybrid models add the branches described above. The click-label vocabulary is constructed from training data only: unseen input labels map to an unknown token, and unseen target labels are counted as incorrect predictions. The primary models do not use class weighting or page-based candidate masking.

The fitted classifiers minimize cross-entropy with Adam (learning rate 0.003 for sequence models and 0.01 for logistic regression; weight decay 0.0001), batch size 256, and gradient-norm clipping at 1. Training lasts at most 30 epochs. Early stopping monitors validation cross-entropy with patience 5 and minimum improvement 0.0001; the best validation checkpoint is restored. Logistic regression uses one-hot encodings of the last three clicks and log-transformed prefix-label counts. The configuration was fixed for this reanalysis rather than selected on the test set; validation data select checkpoints and the UNCP 5 fusion rule, as described above. The fixed user split uses seed 20261103, and neural fits use seeds 11, 23, and 37. Runs used Python 3.12.0, PyTorch 2.11.0, and NumPy 2.2.5. These are revised reference implementations, not exact reconstructions of the original Keras runs; the experiment notebooks and configuration file provide the executable details.

4.7 Quantitative Evaluation Results

The same held-out users and 10,833 prediction targets are used for every method in Table 1. Fitted neural models and the adaptive ensemble report the mean of seeds 11, 23, and 37. In descending Top-3 order, the proposed variants are UNCP 5 (85.09%), UNCP 6 (84.08%), UNCP 2 (83.58%), UNCP (82.30%), UNCP 3 (81.40%), and UNCP 4 (81.08%). The table is ordered by increasing Top-3 accuracy so that the highest result appears last. UNCP 5's paired gain over UNCP 2 is 1.51 percentage points, with an exploratory user-cluster bootstrap interval of 1.03–1.77 points over 11 held-out users. Gate coefficients were selected only on validation users.

Table 2 provides the second and final result table. The temporal subset is a separate robustness analysis with a different eligible cohort, not a directly comparable improvement over the user-disjoint split. UNCP 2–6 have not been evaluated on that subset; their temporal cells are intentionally left unreported. The figure retains the visual comparison used in the original article while replacing the historical contaminated-split scores with the new protocol results.

Top-1 requires the observed next click to be the highest-ranked prediction, whereas Top-3 counts success when it appears among the three highest scores. Both are reported because Top-3 is useful for ranking candidate interactions, but it must be interpreted alongside transparent baselines and the candidate space. Numerical

Table 1: Held-out account comparison, sorted by increasing Top-3 accuracy (percent; fitted models report mean ± sample SD over three seeds).

Model	Top-1	Top-3
Uniform expected	1.10	3.30
Global popularity	5.16	22.37
Logistic regression	44.24 ± 0.61	71.58 ± 0.33
Transformer	44.46 ± 0.52	71.92 ± 0.36
RNN	50.22 ± 0.26	75.10 ± 0.48
LSTM	54.52 ± 0.06	78.00 ± 0.55
GRU	55.97 ± 0.62	78.88 ± 0.40
Markov-4	60.27	79.91
UNCP 4 (typed graph attention, GRU, transition graph, candidate masking, adaptive fusion)	59.84 ± 1.40	81.08 ± 0.88
UNCP 3 (GRU, Transformer, Mamba-2-inspired SSD)	61.12 ± 2.29	81.40 ± 0.82
UNCP (GRU–Transformer fusion)	62.72 ± 0.95	82.30 ± 0.52
UNCP 2 (transition-graph-aware GRU–Transformer)	64.62 ± 1.10	83.58 ± 0.04
UNCP 6 (dual GRU streams, run lengths, repeat/switch head)	66.60 ± 0.51	84.08 ± 0.22
UNCP 5 (UNCP 2, transition model, adaptive fusion)	**66.71** ± 0.84	**85.09** ± 0.20

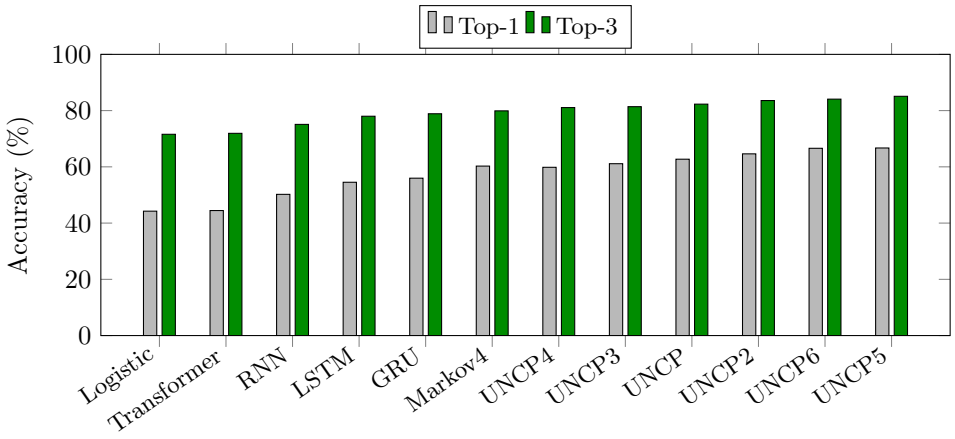


Fig. 10: Updated Top-1 and Top-3 comparison on the user-disjoint test set. Values are generated directly from the verified result files.

Table 2: Top-3 accuracy under two leakage-aware protocols, ordered by increasing held-out-user accuracy (percent). A dash denotes an unrun temporal experiment.

Model	Held-out users	Temporal subset
Transformer	71.92	73.59
RNN	75.10	—
LSTM	78.00	83.84
GRU	78.88	84.74
Markov-4	79.91	88.15
UNCP 4 (typed graph attention, GRU, transition graph, candidate masking, adaptive fusion)	81.08	—
UNCP 3 (GRU, Transformer, Mamba-2-inspired SSD)	81.40	—
UNCP (GRU–Transformer fusion)	82.30	**89.95**
UNCP 2 (transition-graph-aware GRU–Transformer)	83.58	—
UNCP 6 (dual GRU streams, run lengths, repeat/switch head)	84.08	—
UNCP 5 (UNCP 2, transition model, adaptive fusion)	**85.09**	—

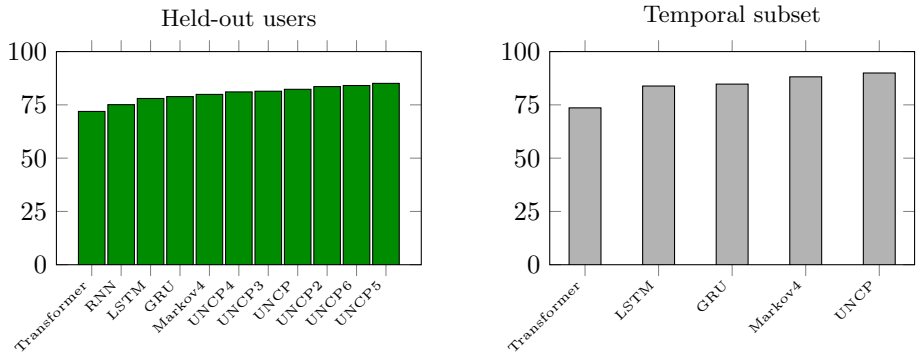


Fig. 11: Top-3 accuracy for selected models under the held-out-user and qualified temporal protocols.

comparisons with other publications are not used to claim superiority because their datasets, interfaces, labels, and split procedures differ.

4.8 User Interface Elements

The dataset contains a variety of user interface elements, including a total of 65 clickable items distributed across 15 pages. These elements include buttons, search fields, text fields, icons, modals, sorting options, filters, and tags. The results, which show the

frequency of clicks on each element across the interface pages, indicate that the most frequently clicked items were found in the *Organizational Thermometer* and *Employee Status* sections under Human Resources. In contrast, the least clicked elements were located in the *Wallet* and *Sorting* sections.

The supplied analytic export contains 93 distinct click-label strings and 19 recorded page strings; these strings are not a verified inventory of unique widgets or screens. The earlier 65-element/15-page design description is therefore retained as application context pending reconciliation with the source interface inventory. Page-based analyses are retrospective and no future destination-page label is supplied to the prediction models.

4.9 Proposed UX Improvements

The predicted rankings and observed click patterns motivated possible changes to the ‘Settings,‘ ‘Home,‘ ‘Marketplace,‘ and ‘Users‘ pages. These included prioritizing frequently used Settings items, reconsidering the order of Home navigation icons, adding purchase-based sorting on Marketplace, and providing a coin-balance ordering option on Users. These proposals illustrate how prediction may inform design review; the study did not measure whether they improved usability or reduced evaluation effort.

The archived clickstream contained frequent clicks on the ‘Close‘ icon in the status-entry menu of the organizational thermometer. This pattern suggests that repeated presentation of the menu merits investigation. Possible design responses include a ‘Do not show again‘ option, a temporary dismissal period, or administrator-configurable display timing. The click log alone does not establish that the menu caused annoyance or that these changes would improve the experience.

The toolbar also allows status registration at any time. This observation may inform a review of early-session interaction and feature visibility, but the available data do not measure attention, loyalty, satisfaction, or the reason a user leaves the application.

These proposed improvements are informed by observed click patterns. Recorded outcomes from the stated retest are unavailable; therefore, this study does not claim that task success, satisfaction, sequence length, or usability improved. A prospective controlled user study is required to evaluate those outcomes.

Future evaluation should test the proposed Settings ordering, Home navigation, Marketplace sorting, Users ordering, and thermometer dismissal options with comparable tasks on the existing and redesigned interfaces. A controlled study can measure task success, completion time, navigation errors, repeated or backtracking clicks, and validated usability and satisfaction scores. First-click tasks and card-sorting exercises should be conducted as distinct user studies where relevant. This would determine which proposed changes help users and whether model-guided inspection saves researchers time.

5 Conclusion and Future work

This study shows that next-click rankings can be learned from the enterprise application clickstream and used to prioritize interaction paths for UX inspection. The revised primary dataset retains repeated-click behavior and contains 46,194 events in 4,223 sequences. Under the held-out-user protocol, validation-selected UNCP 5 reaches the highest mean Top-3 accuracy, 85.09% over three seeds, followed by repeat-aware UNCP 6 at 84.08%, graph-aware UNCP 2 at 83.58%, the original UNCP at 82.30%, and Markov-4 at 79.91%. The paired UNCP 5–UNCP 2 difference is 1.51 percentage points, with an exploratory held-out-user cluster bootstrap interval of 1.03–1.77 points. UNCP 6 did not improve overall accuracy or switch-target Top-3 relative to UNCP 5. These are demonstrated prediction results. The study does not demonstrate a causal improvement in UX, reduced UX-evaluation time or effort, increased user satisfaction, or scalable replacement of first-click testing, card sorting, and controlled usability evaluation. Such outcomes require a prospective controlled or A/B study that reports task success, completion time, errors, abandonment, and validated usability and satisfaction measures.

This study has several limitations. Generalizability is limited by evaluation on one enterprise Android application and an uneven cohort of 52 recorded User IDs, which are not confirmed to represent 52 distinct people. The user-disjoint evaluation on 11 held-out User IDs tests transfer to unseen accounts within this application only. It does not establish performance for other interfaces, organizations, user populations, devices, operating systems, or application categories. Timestamp and label-quality problems, the absence of recorded post-redesign outcomes, and the lack of a formal controlled UX study further constrain interpretation. Within the available data and resources, this revision prioritized leakage correction, transparent baselines, and uncertainty estimates. Future work should recruit a larger and more diverse cohort, evaluate several application categories and device platforms with a fixed protocol, and include an external-app or leave-one-app-out test. A prospective, ethically approved user study is also needed to assess actual UX outcomes.

Additional information

Acknowledgments

ChatGPT assisted with drafting, language editing, code development, and the preparation of analyses and responses to reviewers. The authors are responsible for verifying the data, methods, results, citations, and final manuscript before submission.

Data Availability Statement

The analytical dataset and its audit file used in the revised evaluation are available in a dedicated GitHub data directory.[3] The accompanying model code and experimental outputs are available in the parent repository.

[3] https://github.com/ArmanMhmdi/UNCPArticle2026/tree/main/03%20Data

Ethics and privacy statement

The original export contained UUID-formatted User IDs and auxiliary fields, including Android advertising/device identifiers and IP addresses. The analytical event table retains numeric User IDs, click labels, pages, and times, but excludes those auxiliary identifiers. Replacing the User ID with a number is pseudonymization, not irreversible anonymization; we have no evidence that the original UUID was cryptographically hashed or encrypted. Research-specific informed consent and prior institutional ethics approval or exemption were not obtained. Because these are workplace behavioral records, employees may have felt observed, and their perception and freedom to decline were not assessed. Institutional and editorial guidance is required before resubmission or release of account-level trajectories.

Disclosure statement

No potential conflict of interest was reported by the author.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

References

[1] Böhmer, M., Hecht, B., Schöning, J., Krüger, A., Bauer, G.: Falling asleep with angry birds, facebook and kindle: a large scale study on mobile application usage. In: Proceedings of the 13th International Conference on Human Computer Interaction with Mobile Devices and Services, pp. 47–56 (2011)

[2] Parate, A., Böhmer, M., Chu, D., Ganesan, D., Marlin, B.M.: Practical prediction and prefetch for faster access to applications on mobile phones. In: Proceedings of the 2013 ACM International Joint Conference on Pervasive and Ubiquitous Computing, pp. 275–284 (2013)

[3] Findlater, L., Moffatt, K., McGrenere, J., Dawson, J.: Ephemeral adaptation: The use of gradual onset to improve menu selection performance. In: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, pp. 1655–1664 (2009)

[4] Tanvir, E., Bunt, A., Cockburn, A., Irani, P.: Improving cascading menu selections with adaptive activation areas. International journal of human-computer studies **69**(11), 769–785 (2011)

[5] Lee, S., Ha, R., Cha, H.: Click sequence prediction in android mobile applications. IEEE Transactions on Human-Machine Systems **49**(3), 278–289 (2019) https://doi.org/10.1109/THMS.2018.2868806

[6] Mao, J., Luo, C., Zhang, M., Ma, S.: Constructing click models for mobile search. In: The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval, pp. 775–784 (2018)

[7] Liu, Q., Yu, F., Wu, S., Wang, L.: A convolutional click prediction model. In: Proceedings of the 24th ACM International on Conference on Information and Knowledge Management, pp. 1743–1746 (2015)

[8] Wu, C., Wu, F., Ge, S., Qi, T., Huang, Y., Xie, X.: Neural news recommendation with multi-head self-attention. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp. 6389–6394 (2019)

[9] Browne, D.: Adaptive User Interfaces: Principles and Practice, 1st edn. Elsevier, Amsterdam, Netherlands (2016)

[10] Zhou, X., Li, Y.: Large-scale modeling of mobile user click behaviors using deep learning. In: Proceedings of the 15th ACM Conference on Recommender Systems, pp. 473–483 (2021)

[11] Chen, Q., Zhao, H., Li, W., Huang, P., Ou, W.: Behavior sequence transformer for e-commerce recommendation in alibaba. In: Proceedings of the 1st International Workshop on Deep Learning Practice for High-dimensional Sparse Data, pp. 1–4 (2019)

[12] Qin, J., Zhang, W., Wu, X., Jin, J., Fang, Y., Yu, Y.: User behavior retrieval for click-through rate prediction. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 2347–2356 (2020)

[13] Yang, B., Wei, L., Pu, Z.: Measuring and improving user experience through artificial intelligence-aided design. Frontiers in Psychology **11**, 595374 (2020)

[14] Wu, J., Swearngin, A., Zhang, X., Nichols, J., Bigham, J.P.: Screen correspondence: Mapping interchangeable elements between uis. arXiv preprint arXiv:2301.08372 (2023)

[15] Sun, L., Chen, X., Chen, L., Dai, T., Zhu, Z., Yu, K.: Meta-gui: Towards multi-modal conversational agents on mobile gui. arXiv preprint arXiv:2205.11029 (2022)

[16] Wu, J., Krosnick, R., Schoop, E., Swearngin, A., Bigham, J.P., Nichols, J.: Never-ending learning of user interfaces. In: Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, pp. 1–13 (2023)

[17] Feng, S., Chen, C., Xing, Z.: Video2action: Reducing human interactions in action

annotation of app tutorial videos. In: Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, pp. 1–15 (2023)

[18] Feng, S., Ma, S., Wang, H., Kong, D., Chen, C.: Mud: Towards a large-scale and noise-filtered ui dataset for modern style ui modeling. In: Proceedings of the CHI Conference on Human Factors in Computing Systems, pp. 1–14 (2024)

[19] Zhou, W.-J., Zheng, Y., Feng, Y., Ye, Y., Xiao, R., Chen, L., Yang, X., Xiao, J.: Encode: Breaking the trade-off between performance and efficiency in long-term user behavior modeling. IEEE Transactions on Knowledge and Data Engineering (2024)

[20] Addula, S.R., Perugu, P., Kumar, M.K., Kumar, D., Ananthan, B., Rajasree, R., Sureshkumar, P., Sreetha, G.: Dynamic load balancing in cloud computing using hybrid kookaburra-pelican optimization algorithms. In: 2024 International Conference on Augmented Reality, Intelligent Systems, and Industrial Automation (ARIIA) (2024). https://doi.org/10.1109/ARIIA63345.2024.11051893

[21] Meduri, K., Nadella, G.S., Gonaygunta, H., Kumar, D., Addula, S.R., Satish, S., Maturi, M.H., Rehman, S.U.: Human-centered ai for personalized workload management: A multimodal approach to preventing employee burnout. Journal of Infrastructure, Policy and Development **8**(9), 6918 (2024) https://doi.org/10.24294/jipd.v8i9.6918

[22] Falkowska, J., Sobecki, J.: Replication of first-click eye tracking a/b test of webpage interactive elements. In: FedCSIS (Communication Papers), pp. 145–151 (2022)

[23] Nawaz, A.: A comparison of card-sorting analysis methods. In: APCHI'12. Proceedings of the 10th Asia Pacific Conference on Computer-Human Interaction, pp. 583–592 (2012). Association for Computing Machinery